Word2Vec Explanation

Research pertaining to Word2Vec, as relevant to FYP

Daniel Rice

Word2Vec

A modern (but still traditional[1]) method for forming an embedding for a given corpus is Word2Vec, this method allows for a more- contextually aware embedding, capturing semantic relationships between words [2]. This method maps each word to an N-Dimensional vector[2] with the distance[3] between words representing their relationship, for example the distance between 'boy' and 'girl' should be similar to 'man' and 'woman', assuming the semantic similarity is captured correctly by Word2Vec (i.e. balanced training data). It is worth noting that more dimensions generally refers to a greater-degree of accuracy, however too much *can* result in a decline in accuracy, and is slower to compute (more information to process) (Figure 2 from [4])

Word2Vec captures semantic relationships by way of one of two methods: Continuous Bag of Words (CBoW); or Skip-Gram.

Continuous Bag of Words

Where BoW relies on frequency of words, CBoW uses a sliding context window around a missing word to determine the likely word to predict, thereby taking order and the context of neighbouring words into account, unlike BoW – in this way, CBoW is a target-word- prediction model. CBoW is trained by adjusting the weights in a simple two-layer neural network [1], given an input (missing (target) word's surrounding (context) words), and expected output: the missing word. The more context words surrounding the missing word are captured during the training process (higher context-window size), the greater the context captured[4], however the greater the computational expense. Once the network weights have been adjusted correctly (trained), the aim is to extract the weights 'learned', as this serves as a mapping from the aforementioned one-hot-encoded words (sparse representation) to a dense embedding, which differs from the previous BoW sparse vector (by way of one-hot encoding) in that a dense vector (using weights learned from this method) is a more-compact vector, carrying additional properties such as the typical context learned, or other meaning (when used among other words as context). Therefore, a dense embedding allows words in a similar context (e.g. synonyms) to be 'close' (in N-dimensional space – N defined by the dimension hyperparameter when using CBoW, as aforementioned). At this point, a word (perhaps from a different corpus) can be looked up in the embedding matrix (note the word to look up must have been present during the training process (with sufficient context, for accurate results) to look it up in the embedding matrix), thereby showing typical context for the word looked up. It is because of this simple look-up operation that (after initial training) CBoW is very computationally fast, when used as a method for Word2Vec.

Skip-Gram

Skip-gram is the opposite of CBoW, in that where CBoW trains by predicting the missing word based on the words surrounding it (context window), skip-gram trains by predicting surrounding words (now the missing words) from the target word (previously the missing word in CBoW). This, as expected, is more complex (allowing for more complex relationships between words to be captured), however increasing time-to-train over CBoW, since given less information (singular target word) predict its surrounding (contextually-accurate) words (more information, or at least much-more information, as output, compared with CBoW – dependent on the number of surrounding words to

[1] Word2Vec uses models which are shallow, two-layer neural networks [1]; much-less advanced than, e.g., the transformer architecture (modern deep-learning architecture, presently SoTA architecture for many tasks).

[2] N, the dimensionality, is a hyperparameter which defines the total number of features encoded in the word-vector representation [3]. Greater dimensionality ultimately allows each word-vector to contain more information; greater relationship detail, however the higher N, the higher the computational cost.

[3] Either Euclidean or (more commonly) cosine distance.

[4] However, given a very-high context-window size, as the distance from the missing/target word increases, weighting should decrease [5], since going many words away from the target suggests the distant-word's context is less (or perhaps not at all) relevant to the target-word's context.

predict). During the skip-gram training process, alike CBoW, more context words can be taken as input to increase the quality of the resulting embedding [5], this does, however, further increase computational expense compared with CBoW. However, skip-gram can be advantageous over CBoW, for example the resulting embedding of scarcely-seen words (in the training corpus) will be of higher quality (in terms of contextual capture) over CBoW. Since in CBoW, for a given word, its neighbours, within the context-window's embeddings, are averaged (or summed), and their embeddings will be more frequent (relative to the rare word, hence scarcely appearing), thereby resulting in less contextual capture for the rare word, because its context will be dominated by more-frequent words, as such effectively crushing the less-frequent words. The above can be demonstrated by showing the equations for embedding aggregation of neighbouring contextual words in CBoW; the below two equations show how the embeddings of the neighbouring, more frequent (relative to the target word), words will cause the rarer word to have less contextual meaning in the resultant neural-network weights, and, by extension, in the extracted dense- input-embedding look-up matrix:
Embedding aggregation via summation:

$$\sum_{n=1}^{2n} E_n \cdot W_n$$

Conversely, embedding aggregation via averaging:

$$\frac{1}{2n} \sum_{n=1}^{2n} E_n \cdot W_n$$

Where n represents the neighbour (i.e., if context window is 3, use 3 words either side of the target's (6 total) embedding for context of the target, so $n = 1$ is the first neighbour to sum up to **and including** $2n$, 6 in this example), and E_n represents the embedding of the n^{th} neighbour, and W_n represents the weighting of the n^{th} neighbour, which is controlled by the distance from the target word, i.e. more-distant words area multiplied by a lower weighting, thereby applying less of an impact on the target.
W may look like [0.4, 0.6, 0.8, 0, 0.8, 0.6, 0.4], where the context-window size here is 3, so total 6 contextual words for the target (which is denoted by 0; the target word is not available as such receives no weighting via the 0 in the weight array). In this way, words further from the target are given a lower weighting since they likely have lower impact on the target the further away they are, as aforementioned.
On the other hand, skip-gram is better-suited for infrequent words, this is due to the nature of CBoW against skip-gram – I refer to the target (in CBoW) consisting of its neighbour's embeddings, contrasting to skip-gram, wherein every word (frequent or infrequent) must predict its neighbour's words, thereby ensuring that the more-frequent words (in skip-gram) do not (/cannot) impact less-frequent words. Clearer said, in CBoW, an infrequent word's semantic meaning is largely contaminated with more-frequent words, due to using the more-frequent words to predict the rare word, but as skip-gram performs in the reverse fashion to CBoW, it analyses every word, using even the infrequent words to predict, taking longer to train, however.

References

[1] Wikipedia contributors, "Word2vec — Wikipedia, the free encyclopedia," https://en.wikipedia.org/w/index.php?title=Word2vec&oldid=1301643352, 2025, [Online; accessed 23-July-2025]
In event of dead-link, or changed content, visit: https://web.archive.org/web/20250723154104/https://en.wikipedia.org/w/index.php?title=Word2vec&oldid=1301643352.

[2] GeeksforGeeks contributors, "Word embedding using word2vec — GeeksforGeeks," https://www.geeksforgeeks.org/python/python-word-embedding-using-word2vec/, 2025, [Online; accessed 23-July-2025]
In event of dead-link, or changed content, visit: https://web.archive.org/web/20250723135414/https://www.geeksforgeeks.org/python/python-word-embedding-using-word2vec/.

[3] M. Zhaozhen Xu, "Dimensionality of word embeddings — Baeldung," https://www.baeldung.com/cs/dimensionality-word-embeddings, 2025, [Online; accessed 23-July-2025]
In event of dead-link, or changed content, visit: https://web.archive.org/web/20250723141436/https://www.baeldung.com/cs/dimensionality-word-embeddings.

[4] P. Kevin Patel, "Towards lower bounds on number of dimensions for word embeddings — aclanthology," https://aclanthology.org/I17-2006.pdf, 2017, [Online; accessed 23-July-2025]
(Base page is https://aclanthology.org/I17-2006/)
In event of dead-link, or changed content, visit: https://web.archive.org/web/20241103093905/https://aclanthology.org/I17-2006.pdf.

[5] T. G. K. Jeffrey Dean, "Efficient estimation of word representations in vector space," https://arxiv.org/pdf/1301.3781, 2013, [Online; accessed 23-July-2025]
In event of dead-link, or changed content, visit: https://web.archive.org/web/20250723173031/https://arxiv.org/pdf/1301.3781.